

WPQ: Product Quantization-Augmented Compression for Efficient Low-bit Representation in LLMs

Anonymous ACL submission

Abstract

Large Language Models (LLMs) have achieved remarkable performance but face significant scalability challenges due to their massive memory requirements. While quantization is a widely used technique to reduce memory overhead, the presence of outliers in LLMs makes low-precision quantization particularly challenging, often leading to accuracy degradation.

In this work, we propose WPQ, a novel low-bit compression framework that leverages a Product Quantization (PQ)-augmented approach for efficient weight compression. WPQ leverages PQ’s inherent robustness to outliers, combined with SmoothQuant (Xiao et al., 2023), to shift the quantization difficulty from activations to weights. By applying PQ to weights, WPQ achieves high compression ratios while maintaining accuracy, even at low precision. Specifically, WPQ with W4A8 quantization achieves comparable performance to W4A16 quantization using AWQ (Lin et al., 2024a), demonstrating its effectiveness in balancing compression and accuracy. To further enhance efficiency, we introduce shared codebooks, enabling identical structures across layers to reuse the same codebook. This reduces storage overhead and improves compression without sacrificing performance. Our experiments show that WPQ provides a robust and hardware-friendly solution for outlier-resilient, low-precision quantization, pushing the boundaries of LLM compression while minimizing performance degradation.

1 Introduction

Large language models (LLMs) have demonstrated substantial progress, exhibiting exceptional performance across various tasks and significantly influencing our daily routines (Zhao et al., 2023; Zhang et al., 2022). However, their enormous size leads to high operational costs, excessive energy consumption, and significant inference latency, posing challenges for real-world deployment. For instance, the

Llama-3.1-405B model (Dubey et al., 2024)—with 405 billion parameters—requires at least 810GB of memory in FP16, necessitating a cluster of at least eleven 80GB A100 GPUs for inference. This highlights the urgent need for efficient optimization strategies to reduce LLM computational overhead.

Recently, weight quantization has emerged as a promising solution to alleviate these challenges (Dettmers et al., 2022; Xiao et al., 2023). By storing parameters in low-precision formats, quantization significantly reduces memory consumption and alleviates memory bottlenecks, improving efficiency in model deployment and inference. Studies have demonstrated that lowering both weight and activation precision can substantially reduce GPU memory requirements while enhancing computational throughput. For example, INT8 quantization can halve memory usage and accelerate matrix multiplications compared to FP16.

However, as LLM scales increase, systematic outliers emerge in activations, posing a major challenge for low-bit quantization (Dettmers et al., 2022). These extreme values amplify quantization error, hindering accuracy in lower-precision formats. Existing methods address this issue in different ways: LLM.int8() (Dettmers et al., 2022) adopts mixed-precision quantization to mitigate outlier effects. SmoothQuant (Xiao et al., 2023), on the other hand, transfers outliers from activations to weights through a mathematically equivalent transformation, simplifying weight quantization. While effective in some cases, these approaches struggle at lower-bit precision such as W4A8 and W4A4, where outliers still degrade model performance. Thus, designing robust outlier-aware quantization techniques remains a crucial challenge for enabling ultra-low precision LLM deployment.

In this work, we propose WPQ, an accurate post-training quantization method for LLMs. WPQ is based on the key observation that Product Quantization (PQ) is highly resilient to outliers. It achieves

this by confining the impact of outliers within individual subspaces, dispersing reconstruction errors across subspaces, and exploiting the statistical stability of lower-dimensional subspaces. Building on this insight, we combine SmoothQuant to shift quantization difficulty from activations to weights, while utilizing PQ’s outlier resilience to perform lower-precision weight quantization with minimal performance degradation. To reduce the storage overhead of PQ codebooks, we introduce a shared codebook approach, where weights from the same parts of different layers (e.g., query matrix weights in LLaMA) share the same codebook. This strategy not only minimizes storage costs but also increases training samples per codebook, enhancing quantization robustness. Our experiments show that WPQ, using W4A8 quantization, achieves accuracy comparable to AWQ with W4A16 quantization, and in most cases, slightly outperforms state-of-the-art QoQ with W4A8 quantization. The contributions of our work are summarized as follows:

- We propose WPQ, a post-training quantization method that leverages the outlier resilience of PQ to enable low-precision weight quantization while maintaining model accuracy.
- We introduce a shared codebook approach, where weights from similar parts of different layers (e.g., query matrices in LLaMA) share the same codebook. This reduces storage overhead and improves quantization robustness by increasing the number of training samples per codebook.
- We demonstrate the effectiveness of WPQ through extensive experiments, showing that WPQ with W4A8 quantization achieves comparable accuracy to AWQ (W4A16) and often outperforms the state-of-the-art QoQ (W4A8) across various tasks and models.

2 Preliminary

In this section, we present the background for our approach about LLM quantization.

Large Language Models. Large Language Models (LLMs) are advanced neural network architectures designed to process and generate natural language text with high accuracy and fluency. These models are typically composed of multiple identically-structured layers, each comprising an attention block, a feed-forward network (FFN), and

normalization layers. The LLM inference process consists of two phases: the prefilling stage processes all input prompts in parallel to establish context, while the decoding stage generates output tokens sequentially — the former is computationally intensive requiring full prompt token processing, and the latter is memory-bound primarily involving repetitive reads of model parameters.

General Quantization Method. Quantization is a widely adopted optimization technique designed to reduce the memory footprint of models by converting the model parameters from high-precision representations (such as 32-bit floating point) to lower-precision formats. The key advantage of quantization lies in its ability to significantly reduce the model size, which in turn enables faster storage, transmission, and more efficient use of hardware resources. Beyond size reduction, quantization can also lead to substantial performance gains by allowing the use of low-precision arithmetic operations, which are more computationally efficient on modern hardware accelerators, such as GPUs and TPUs.

The process of quantization can typically be described mathematically by the following formula:

$$X_q = \left\lfloor \frac{X}{s} + z \right\rfloor \quad (1)$$

$$\hat{X} = (X_q - z) \cdot s \quad (2)$$

where X is floating point tensor, X_q is its n -bits quantized counterpart, s is the scaling factor and z is the zero point, and we can obtain the dequantized tensor $\hat{X}$ through Eq. 2. For simplicity, we use symmetric quantization where $z = 0$ and $s = \frac{\max(|X|)}{2^{n-1}-1}$. However, the existence of outliers, which significantly deviate from the majority of values (Dettmers et al., 2022), makes our quantization error considerable.

Quantization for LLMs. LLM.int8() (Dettmers et al., 2022) and SmoothQuant (Xiao et al., 2023) are two significant techniques developed for quantizing large language models (LLMs), both addressing the challenge of outliers to achieve higher compression ratios and efficient inference. LLM.int8() reduces the model size considerably by quantizing both weights and activations to 8-bit precision. To handle outliers, LLM.int8() employs a mixed-precision decomposition strategy, isolating the few abnormal feature dimensions into 16-bit matrix

multiplications, while the majority of values are processed at 8-bit precision. This approach enables significant size reduction without compromising performance on the majority of model parameters. SmoothQuant, on the other hand, leverages the relatively concentrated distribution of weights in LLMs by applying a mathematically equivalent identity transformation. This transformation shifts outliers from activations to weights, which reduces the quantization error by better aligning the outliers with the structure of the weight matrix. The transformation can be expressed as:

$$Y = (X \text{diag}(\Delta)^{-1}) \cdot (\text{diag}(\Delta)W) = \hat{X}\hat{W} \quad (3)$$

where Δ is the smoothing factor, which can be fused into the parameters of previous layers offline, allowing for efficient computation. While these methods effectively mitigate the impact of outliers, they tend to introduce larger quantization errors, particularly when attempting to achieve higher compression ratios.

3 Methods

In this section, we introduce an effective *Product Quantization-Augmented* method targeting *Weights*, denoted WPQ, to address the quantization error caused by outliers.

3.1 Analysis of Quantization Difficulty for LLMs

Unlike smaller models such as CNNs or BERT (Kenton and Toutanova, 2019), the quantization of large language models (LLMs) faces significant challenges due to the presence of outliers. As model size increases, the impact of outliers becomes more pronounced, resulting in larger quantization errors and considerable accuracy degradation (Dettmers et al., 2022; Bondarenko et al., 2021). We identify several key patterns that highlight the challenges of LLM quantization:

Outliers Lead to the Quantization Difficulty of Activations. The occurrence of outliers within activation values complicates the quantization process and can result in substantial quantization errors (Bondarenko et al., 2021; Wei et al., 2022). Notably, the magnitude of these outliers can be approximately $\sim 100\times$ greater than the typical activation values. Under per-tensor quantization schemes, such significant outliers predominantly influence the determination of maximum magnitudes. Consequently, this leads to non-outlier channels having

substantially reduced effective quantization levels, which in turn, induces substantial quantization errors for these channels. Research has indicated that the distribution of weights in Large Language Models (LLMs) tends to be more uniform compared to that of activations, facilitating a more straightforward quantization process. Typically, quantizing the weights of LLMs to INT8 does not incur significant accuracy loss (Dettmers et al., 2022). Based on these observations, SmoothQuant (Xiao et al., 2023) is proposed. As shown in Figure. 1, SmoothQuant employs mathematical equivalence transformations to shift the quantization challenges from activations to weights, thereby facilitating effective quantization of both. Under a W8A8 quantization scheme, the impact on model performance is minimal.

The Difficulty of Low-Precision Quantization.

While SmoothQuant helps mitigate activation quantization challenges by transferring outliers to weights, it faces significant difficulties when quantizing weights to a lower bit-width, such as W4A8. At this precision level, the limited bit-width exacerbates the impact of outliers, resulting in increased quantization errors. Table 1 highlights that after transferring outliers to weights, they become more challenging to quantize, leading to noticeable performance degradation at W4A8 precision. To overcome this, a more robust algorithm specifically designed for weight quantization is needed. Recent work (Ashkboos et al., 2024), (Lin et al., 2024b), and (Zhao et al., 2024) has addressed outlier impact through techniques like rotation, reordering, and pruning.

Building on this, we propose a novel algorithm—Weight Product Quantization (WPQ)—which is specifically designed to mitigate outlier effects in weight quantization. This approach is fully compatible with existing quantization methods, enabling their synergistic combination to further optimize performance through complementary mechanisms.

3.2 WPQ: Weight Product Quantization

3.2.1 Product Quantization (PQ)

Product Quantization (PQ) is an advanced quantization technique that efficiently compresses high-dimensional vectors by decomposing the vector space into multiple lower-dimensional subspaces and quantizing each subspace independently. A product quantization scheme encodes each vector into a compressed form as shown in Figure 2. The

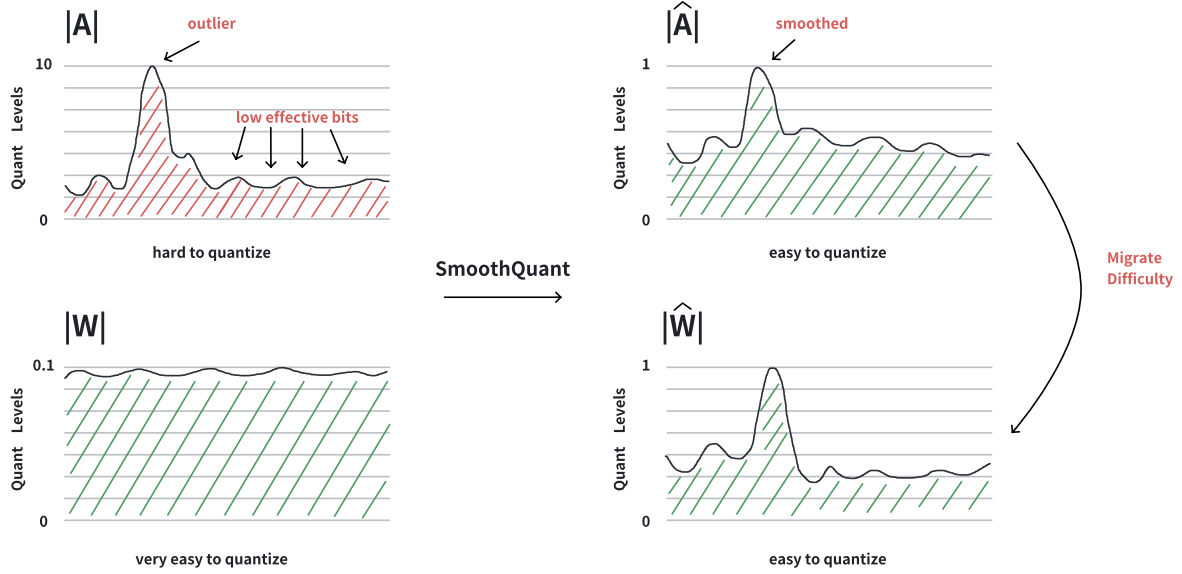


Figure 1: The Process of SmoothQuant.

Table 1: Perplexity on WikiText-2 with SmoothQuant at Various Precision Levels.

WikiText2 Perplexity ↓		Llama-3	Llama-2		Llama		
Precision	Algorithm		7B	13B	7B	13B	30B
FP16	—	6.14	5.47	4.88	5.68	5.09	4.10
W8A8	SmoothQuant	6.28	5.54	4.95	5.73	5.13	4.23
W4A8	SmoothQuant	11.52	6.98	5.55	7.12	5.88	5.37

process can be illustrated as follows:

Subspace Decomposition. For the purpose of dimensionality reduction to facilitate subsequent advanced compression, we first partition a vector into multiple sub-vectors. For instance, consider a vector x that is partitioned into M sub-vectors $x_0, x_1, \dots, x_{M-1}$, where x is a D -dimensional vector and each x_i has a dimension of D/M . As shown in Figure 2, we assume $D = 6$ and $M = 3$, so each sub-vector is a 2-dimensional vector. Subsequently, to derive the identifiers corresponding to each sub-vector, it is necessary to establish a codebook for each subspace. A codebook, in essence, constitutes a collection of representative vectors. Prior research has introduced diverse algorithms (Johnson et al., 2021; Ge et al., 2014) for constructing these codebooks. Following an independent training procedure for each subspace, we acquire the codebooks $B_0, B_1, \dots, B_{M-1}$. Each codebook B_i corresponds to a D/M -dimensional subspace, and comprises k codeword vectors, with each codeword vector maintaining a dimensionality of D/M .

In the figure, we illustrate an example with $M = 3$ codebooks, each containing $k = 4$ codeword vectors. Each codebook B_i comprises codeword vectors $B_i[0], \dots, B_i[k-1]$.

Similarity Computation and Encoding. Then, the sub-vector x_i is encoded into an identifier by selecting the nearest codeword from the codebook B_i , based on similarity. The similarity is typically computed using either the inner product or the L2 distance. Specifically, x_i is represented as $e_i(x) = j$, where j is the index of the selected codeword $B_i[j]$. By repeating this process for all sub-vectors, the concatenation of the resulting identifiers yields the final quantized representation.

Decoding and Search. In the decoding phase of product quantization, we can obtain the dequantization results by first identifying the encoded identifiers $e_i(x)$ for each subspace. We then look up the corresponding codeword vectors $B_i[e_i(x)]$ from the codebook B_i of each subspace. Finally, we concatenate these codeword vectors to recon-

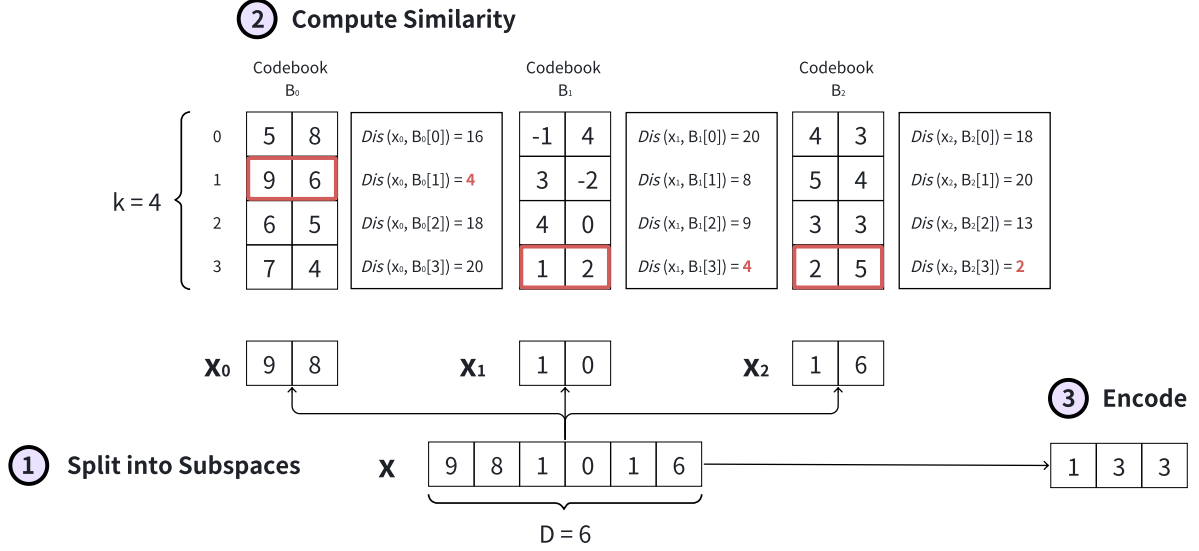


Figure 2: The Process of PQ.

struct the output, effectively reversing the quantization process. During the process of search, each query vector q is decomposed into sub-vectors $q_0, q_1, \dots, q_{M-1}$, corresponding to the subspaces used during encoding. Each sub-vector q_i is then mapped back to its nearest codeword $B_i[e_i(q)]$ in the respective codebook B_i . The reconstructed vector is obtained by concatenating these codewords, yielding an efficient approximation of the original query vector q . By repeating this process across all subspaces, the similarity between the query vector q and a candidate vector x' can be computed as the sum of the similarities between the corresponding sub-vectors q_i and x_i , which can be efficiently approximated using the precomputed codebook representations:

$$Dis_{ip}(q, x') = \sum_{i=0}^{M-1} q_i B_i[e_i(x)] (Inner Product) \quad (4)$$

$$Dis_{L2}(q, x') = \sum_{i=0}^{M-1} \|q_i - B_i[e_i(x)]\|^2 (L2 Distance) \quad (5)$$

To further accelerate the process, we can construct a lookup table for each codebook (Lee et al., 2022). This lookup table stores the precomputed similarities between each codeword and a set of representative query vectors. By leveraging these lookup tables, the similarity computation can be significantly sped up, as it reduces the need for real-time distance calculations.

Due to the distinctive subspace decomposition strategy inherent in PQ, this quantization technique is capable of markedly diminishing the influence of outliers on the overarching quantization procedure, as detailed below (Jegou et al., 2010):

Localization and Isolation Effects. Outliers that may deviate significantly in the overall high-dimensional space are dispersed when partitioned into different subspaces. For instance, a 128-dimensional outlier may appear normal in some subspaces after decomposition. Each subspace’s quantizer focuses solely on local statistical properties. The impact of outliers is thus confined to individual subspaces, preventing their adverse effects from contaminating the global model.

Error Dispersion. Traditional quantization methods are highly sensitive to errors across the entire space. In contrast, PQ distributes reconstruction errors across multiple subspaces. Even if a particular subspace experiences significant error due to outliers, the accurate quantization of other subspaces can partially offset the overall error.

Statistical Stability of Subspaces. When subspaces have lower dimensions, data distributions tend to be more compact, and clustering centers exhibit greater robustness to local noise, including outliers. For example, in a 16-dimensional subspace, the deviation direction of an outlier may be

“diluted” by other dimensions.

Inspired by the above observations, we proposed WPQ to mitigate the quantization difficulty. After transferring the outlier of activation to weight through SmoothQuant, we utilize the anti-outlier ability of PQ to achieve lower precision weight quantization.

3.2.2 Details in WPQ

Since weights can be easily quantized offline, we focus on optimizing weight quantization to enhance the accuracy of low-bit quantization. To mitigate the impact of outliers in activations, SmoothQuant (Xiao et al., 2023) is employed to shift the quantization difficulty from activations to weights. After this process, the proportion of outliers in weights increases, making quantization more challenging. To address this, we leverage the outlier resilience of PQ by partitioning the weights into as many subspaces as possible, thereby reducing the impact of outliers on quantization error. Unlike SmoothQuant, which only applies smoothing to the QKV Projection Layer and the FFN 1st Layer, WPQ extends the smoothing process to include the O Projection Layer and the FFN 2nd Layer as well. Moreover, since the outlier distribution in these layers (e.g., o_proj, down_proj) is less pronounced, the migration intensity varies. Based on experimental results, the migration strength α for these layers is smaller. To further reduce the quantization error caused by outliers, we also applied mixed-precision quantization to the weights (Dettmers et al., 2022).

3.3 Advanced Optimization

The quantization error of PQ primarily depends on two key factors: the number of subspaces and the size of the codebooks for each subspace. Generally, increasing the number of subspaces and the size of the codebooks can enhance quantization accuracy. However, this improvement comes at the cost of increased computational complexity and storage requirements. Specifically, training a separate codebook for each weight matrix would significantly reduce the model’s compression ratio, thereby negating the benefits of quantization. Therefore, in our method, a careful trade-off between quantization accuracy and memory footprints must be considered to optimize overall performance. Since weights from the same parts of different layers share identical shapes and similar functionalities (e.g., q_proj across layers), we use them to train the same set of codebooks. This

approach not only improves the compression rate but also provides more training samples for each codebook, enhancing their robustness.

4 Experiments

In this section, we mainly evaluate WPQ on the language modeling task to compare it with other quantization approaches.

4.1 Experimental Setup

Benchmarks. We evaluated our method on the Llama, Llama-2, Llama-3 families (Touvron et al., 2023a,b; Dubey et al., 2024), as well as the Mistral-7B model (Jiang et al., 2023). Following prior quantization schemes (Dettmers et al., 2022; Frantar et al., 2022; Xiao et al., 2023; Lin et al., 2024a; Ashkboos et al., 2024; Zhao et al., 2024), we evaluated the performance of WPQ-quantized models on the language modeling task. Specifically, we evaluated on WikiText2 (Merity et al., 2016) for perplexity. This is a widely-used language modeling dataset derived from Wikipedia articles, known for its high quality and rich context. Comprising complete articles, it is particularly suitable for models that can leverage long-term dependencies.

Baselines. We compared our method to widely used post-training LLM quantization techniques, SmoothQuant (Xiao et al., 2023), GPTQ (Frantar et al., 2022), AWQ (Lin et al., 2024a), and recently released state-of-the-art 4-bit weight quantization frameworks, Atom (Zhao et al., 2024), QuaRot (Ashkboos et al., 2024), and QoQ (Lin et al., 2024b). SmoothQuant alleviates the quantization difficulty by smoothing activation outliers into the weights. GPTQ leverages Hessian-based optimization to minimize output error by iteratively quantizing weights and compensating for quantization errors. AWQ selectively amplifies significant weights based on activation distributions to reduce quantization error in low-bit scenarios. Atom optimizes 4-bit weight quantization to achieve high precision and reduced model storage for resource-constrained devices. QuaRot uses random Hadamard transforms to rotate model inputs, enabling 4-bit quantization of weights, activations, and KV caches with minimal accuracy loss. QoQ combines progressive grouping quantization and SmoothAttention to optimize W4A8KV4 quantization for high accuracy and efficient inference.

Table 2: WikiText2 perplexity with 2048 sequence length. The lower is the better.

WikiText2 Perplexity ↓		Llama-3	Llama-2		Llama		Mistral
Precision	Algorithm	8B	7B	13B	7B	13B	7B
FP16	–	6.14	5.47	4.88	5.68	5.09	5.25
W8A8	SmoothQuant	6.28	5.54	4.95	5.73	5.13	5.29
W4A16	GPTQ	6.56	5.63	4.99	5.83	5.20	5.39
g128	AWQ	6.54	5.60	4.97	5.78	5.19	5.37
W4A4	QuaRot	8.20	6.10	5.40	6.26	5.55	5.71
		8.33	6.19	5.45	6.34	5.58	5.77
W4A4	QuaRot	7.32	5.93	5.26	6.06	5.40	5.54
g128		7.51	6.00	5.31	6.13	5.43	5.58
	Atom	7.57	6.03	5.27	6.16	5.46	5.66
		7.76	6.12	5.31	6.25	5.52	5.76
W4A8	QoQ	6.59	5.62	5.01	5.80	5.20	5.37
	WPQ	6.59	5.60	5.00	5.72	5.19	5.40

* Grayed results use Wikitext2 as calibration dataset.

4.2 Main Results

WikiText2 Perplexity. Table 2 compares the WikiText2 perplexity results between WPQ and other baselines, where lower values indicate better performance. For the Llama-207B model, WPQ exhibited outstanding performance characteristics, with perplexity rising by only 0.06 compared to the W8A8 SmoothQuant baseline, indicating a negligible effect on model accuracy. Moreover, W4A8 WPQ not only outperformed GPTQ at the W4A16 quantization precision but also matched AWQ in performance. Additionally, WPQ demonstrated significant improvements in perplexity, with up to 0.40 and 0.52 reductions compared to W4A4 QuaRot and Atom, respectively. Notably, at the identical W4A8 precision level, WPQ slightly surpassed the state-of-the-art QoQ in perplexity.

Ablation Study. We evaluated the impact of various quantization techniques within WPQ on perplexity by conducting tests on the Llama-2 7B model using the WikiText-2 dataset. Firstly, at the W8A8 quantization precision, both SmoothQuant and mixed-precision quantization showed remarkable performance, with the perplexity rising by no more than 0.1. However, reducing the weight quantization precision and employing these methods in-

dividually led to substantial accuracy degradation. Particularly, SmoothQuant, which reallocates the quantization difficulty from activations to weights, could result in a perplexity as high as 6.98 when operating at lower weight quantization precision levels. Subsequently, we integrated these methods with product quantization, achieving varying degrees of improvement and even attaining accuracy comparable to that of W8A8 precision. Finally, we refined PQ through the use of shared codebooks, enhancing the compression ratio while endeavoring to mitigate the loss of precision.

5 Related Work

In this section, we present related works as well as draw distinctions of our approach to existing literature.

Large Language Models (LLMs). Pre-trained language models have demonstrated outstanding performance across a diverse range of tasks. Recent progress has seen models grow in scale, with some now exceeding 100 billion parameters (Smith et al., 2022; Brown et al., 2020; Zhang et al., 2022; Le Scao et al., 2023). However, this increase in capability is accompanied by the challenge of escalating memory overhead. In this study, our proposed method effectively quantizes LLMs to mitigate this

Table 3: Ablation study on quantization techniques used in WPQ. The mode used here is Llama-2-7B.

Precision	Algorithm	Perplexity ↓
FP16	–	5.47
W8A8	SmoothQuant	5.54
	Mixed Precision	5.57
W4A8	SmoothQuant	6.98
	Mixed Precision	5.79
	SmoothQuant + Mixed Precision	5.74
	SmoothQuant + PQ	5.68
	Mixed Precision + PQ	5.68
	SmoothQuant + Mixed Precision + PQ	5.57
	SmoothQuant + Mixed Precision + Shared PQ	5.60

memory cost.

Outliers in LLMs. Outliers in LLMs have been extensively studied (Timkey and Van Schijndel, 2021; Wei et al., 2022), and it has been demonstrated that outliers significantly contribute to quantization difficulty, particularly as model sizes increase (Dettmers et al., 2022). Previous research has established a theoretical connection between the emergence of outliers in transformers and factors such as layer normalization and token frequency distribution (Gao et al., 2019). Additionally, (Puccetti et al., 2022) empirically shows that the appearance of outliers is closely tied to the frequency of tokens in the training data distribution. By leveraging PQ’s inherent resilience to outliers, we effectively mitigate their impact on quantization error, achieving higher compression rates while preserving model accuracy.

Quantization for LLMs. Quantization has been proven to be an effective approach for compressing model size, reducing storage consumption, and accelerating inference speed. Classic quantization methods are mainly focused on convolutional neural networks or smaller models like BERT (Kenton and Toutanova, 2019), such as Min-Max quantization (Jacob et al., 2018), Pact (Choi et al., 2018), Q-bert (Shen et al., 2020), Bibert (Qin et al., 2022) and etc. To address the unique challenges posed by large-scale and complex architectures, several quantization methods specifically tailored for LLMs have been proposed. For example, GPTQ (Frantar et al., 2022) employs gradient-based optimization to minimize quantization error during post-training quantization. AWQ (Lin

et al., 2024a) adaptively adjusts the quantization strategy based on activation distributions, using non-uniform quantization and dynamic scaling to reduce errors. QuaRot (Ashkboos et al., 2024) leverages a rotation-based approach to suppress outliers in weights and activations, enabling effective 4-bit quantization while preserving model performance. QoQ (Lin et al., 2024b) optimizes inference speed and precision through progressive group-wise quantization and SmoothAttention for KV cache quantization.

6 Conclusion

We propose WPQ, a novel algorithm designed to quantize Large Language Models (LLMs) to W4A8 precision. WPQ aims to mitigate the impact of outliers on quantization error by leveraging SmoothQuant to shift outliers from activations to weights and utilizing PQ’s outlier resilience to perform low-precision quantization on weights. Through experiments, we demonstrate that WPQ effectively reduces GPU memory usage while maintaining model accuracy as much as possible. WPQ democratizes the application of LLMs by offering a solution to reduce the serving cost.

Limitations

There are two limitations in this study. Firstly, given the varying weight distributions across different models, it is crucial to train the centroid of each model. Secondly, while the shared codebook for weight values has been intricately designed, there is an additional overhead beyond 4-bit quantization, which can be overlooked when dealing with a large number of model parameters.

References

- Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefer, and James Hensman. 2024. Quarot: Outlier-free 4-bit inference in rotated llms. *arXiv preprint arXiv:2404.00456*.
- Yelysei Bondarenko, Markus Nagel, and Tijmen Blankevoort. 2021. Understanding and overcoming the challenges of efficient transformer quantization. *arXiv preprint arXiv:2109.12948*.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. 2018. Pact: Parameterized clipping activation for quantized neural networks. *arXiv preprint arXiv:1805.06085*.
- Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems*, 35:30318–30332.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2022. Gptq: Accurate post-training compression for generative pretrained transformers. *arXiv preprint arXiv:2210.17323*, 1.
- Jun Gao, Di He, Xu Tan, Tao Qin, Liwei Wang, and Tie Yan Liu. 2019. Representation degeneration problem in training natural language generation models. *arXiv preprint*.
- Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2014. Optimized product quantization. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 36(4):744–755.
- Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2704–2713.
- Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence*, 33(1):117–128.
- Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7b. *arXiv preprint arXiv:2310.06825*.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547.
- Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of naacL-HLT*, volume 1. Minneapolis, Minnesota.
- Teven Le Scao, Angela Fan, Christopher Akiki, Elie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, et al. 2023. Bloom: A 176b-parameter open-access multilingual language model.
- Yejin Lee, Hyunji Choi, Sunhong Min, Hyunseung Lee, Sangwon Beak, Dawoon Jeong, Jae W Lee, and Tae Jun Ham. 2022. Anna: Specialized architecture for approximate nearest neighbor search. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 169–183. IEEE.
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024a. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems*, 6:87–100.
- Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. 2024b. Qserve: W4a8kv4 quantization and system co-design for efficient llm serving. *arXiv preprint arXiv:2405.04532*.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*.
- Giovanni Puccetti, Anna Rogers, Aleksandr Drozd, and Felice Dell’Orletta. 2022. Outlier dimensions that disrupt transformers are driven by frequency. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, page 1286–1304. Association for Computational Linguistics.
- Haotong Qin, Yifu Ding, Mingyuan Zhang, Qinghua Yan, Aishan Liu, Qingqing Dang, Ziwei Liu, and Xi-anlong Liu. 2022. Bibert: Accurate fully binarized bert. *arXiv preprint arXiv:2203.06390*.
- Sheng Shen, Zhen Dong, Jiayu Ye, Linjian Ma, Zhewei Yao, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. 2020. Q-bert: Hessian based ultra low precision quantization of bert. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8815–8821.

- Shaden Smith, Mostofa Patwary, Brandon Norick, Patrick LeGresley, Samyam Rajbhandari, Jared Casper, Zhun Liu, Shrimai Prabhumoye, George Zerveas, Vijay Korthikanti, et al. 2022. Using deep-speed and megatron to train megatron-turing nlG 530b, a large-scale generative language model. *arXiv preprint arXiv:2201.11990*.
- William Timkey and Marten Van Schijndel. 2021. All bark and no bite: Rogue dimensions in transformer language models obscure representational quality. *arXiv preprint arXiv:2109.04404*.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023a. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023b. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Xiuying Wei, Yunchen Zhang, Xiangguo Zhang, Ruihao Gong, Shanghang Zhang, Qi Zhang, Fengwei Yu, and Xianglong Liu. 2022. Outlier suppression: Pushing the limit of low-bit transformer language models. *Advances in Neural Information Processing Systems*, 35:17402–17414.
- Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pages 38087–38099. PMLR.
- Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*.
- Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223*.
- Yilong Zhao, Chien-Yu Lin, Kan Zhu, Zihao Ye, Lequn Chen, Size Zheng, Luis Ceze, Arvind Krishnamurthy, Tianqi Chen, and Baris Kasikci. 2024. Atom: Low-bit quantization for efficient and accurate llm serving. *Proceedings of Machine Learning and Systems*, 6:196–209.